SenEvent

apps/web · **apps/mobile** · packages/shared

DAWM · BLOC MOBILE 3/3 · M1 RETEL — DMI/FST/UCAD

Lab 10 — Auth et catalogue complet côté mobile

Rapport de réalisation : l'application mobile, jusqu'ici en lecture seule, sait maintenant se connecter, créer un compte, publier un événement et s'y inscrire. La session est partagée par un contexte React, les formulaires sont natifs, et chaque action appelle les fonctions de `@senevent/shared` déjà utilisées par le web. La démo croisée l'a vérifié sur la vraie base : ce qui est créé d'un côté est vu, et modifié, de l'autre.

tag : v0.10-mobile-crud | Expo SDK 57 · React Native 0.86.3 | AuthProvider + useAuth | TextInput · KeyboardAvoidingView | useFocusEffect | 0 appel Supabase en dur

01 Le mobile rattrape le web

À la fin du Lab 9, l'application mobile savait naviguer mais seulement regarder. Le web, lui, connecte, crée et inscrit depuis les Labs 5 et 6. Toute cette logique existait déjà dans le package partagé : il restait à construire l'interface native par-dessus.

CAPACITE	WEB	MOBILE AU LAB 9	MOBILE AU LAB 10
Se connecter, creer un compte	Lab 5	—	onglet Profil
Voir la liste	Lab 5	Lab 8	Lab 8, rechargee au focus
Voir le detail	Lab 4	Lab 9	Lab 9, avec inscription
Creer un evenement	Lab 6	—	ecran nouveau.js
S'inscrire a un evenement	Lab 6	—	BoutonInscription
Logique metier	**@senevent/shared**	**@senevent/shared**	**@senevent/shared**

La dernière ligne n'a pas bougé de tout le lab : aucune fonction métier n'a été réécrite côté mobile. Le seul ajout au package partagé est `mesInscriptions()`, pour le bonus — et elle est aussitôt disponible pour le web.

02 Le contexte React remplace le passage de props

Au Lab 5, `App.jsx` détenait la session et la passait en prop à `NavBar` et `Detail`. Avec Expo Router, les écrans sont rendus par le routeur : l'onglet Profil et l'écran de détail n'ont pas de parent commun qui pourrait la leur transmettre. Le contexte rend la session disponible à toute une sous-arborescence, et n'importe quel écran la lit avec `useAuth()`.

PIECE	ROLE	DANS SENEVENT
`createContext`	cree le contexte et sa valeur par defaut	{ session: null, chargement: true }
`<Context.Provider>`	fournit la valeur a toute la sous-arborescence	AuthProvider, autour du Stack racine
`useContext`	lit la valeur depuis n'importe quel descendant	useAuth() : 4 fichiers l'appellent

APPS/MOBILE/CONTEXT/AUTHCONTEXT.JS

```
const AuthContext = createContext({ session: null, chargement: true });

export function AuthProvider({ children }) {
  const [session, setSession] = useState(null);
  const [chargement, setChargement] = useState(true);

  useEffect(() => {
    // Recuperer la session au demarrage
    getSupabase()
      .auth.getSession()
      .then(({ data }) => {
        setSession(data.session);
        setChargement(false);
      });

    // Ecouter les changements (login, logout, refresh du token)
    const { data: sub } = getSupabase().auth.onAuthStateChange(
      (_event, nouvelleSession) => setSession(nouvelleSession)
    );

    return () => sub.subscription.unsubscribe();
  }, []);

  return (
    <AuthContext.Provider value={{ session, chargement }}>
      {children}
    </AuthContext.Provider>
  );
}

// Hook pratique pour lire la session depuis n'importe quel ecran
export function useAuth() {
  return useContext(AuthContext);
}
```

C'est exactement la logique de session du Lab 5 — `getSession`, `onAuthStateChange`, désabonnement au démontage — rangée dans un contexte plutôt que dans `App.jsx`. Le layout racine enveloppe tout le Stack, et déclare au passage l'écran de création :

APPS/MOBILE/APP/_LAYOUT.JS

```
<AuthProvider>
  <StatusBar style="light" />
  <Stack
    screenOptions={{ /* theme sombre, inchange depuis le Lab 9 */ }}
  >
    <Stack.Screen name="(tabs)" options={{ headerShown: false }} />
    <Stack.Screen name="evenement/[id]" options={{ title: "Detail" }} />
    <Stack.Screen name="nouveau" options={{ title: "Nouvel evenement" }} />
  </Stack>
</AuthProvider>
```

03 L'onglet Profil : connexion, inscription, déconnexion

L'écran a trois états, choisis d'après le contexte : session en chargement (indicateur), connecté (le compte et le bouton de déconnexion), déconnecté (le formulaire). Le passage de l'un à l'autre n'est jamais provoqué par l'écran lui-même : après `seConnecter()` ou `seDeconnecter()`, c'est `onAuthStateChange` qui met le contexte à jour, et tous les écrans abonnés se redessinent.

PROP DE TEXTINPUT	EQUIVALENT WEB	UTILISEE POUR
`value` + `onChangeText`	value + onChange	champ controle ; onChangeText donne le texte, pas un evenement
`secureTextEntry`	type="password"	masquer le mot de passe
`keyboardType="email-address"`	type="email"	clavier avec @, sans majuscule (autoCapitalize)
`keyboardType="number-pad"`	type="number"	le prix, entier
`onSubmitEditing`	submit du <form>	la touche Entree du clavier envoie le formulaire

LE CHAMP MOT DE PASSE

```
<TextInput
  style={styles.input}
  placeholder="Mot de passe"
  placeholderTextColor={couleurs.texteFaible}
  secureTextEntry
  textContentType={inscription ? "newPassword" : "password"}
  returnKeyType="go"
  onSubmitEditing={soumettre}
  value={motDePasse}
  onChangeText={setMotDePasse}
/>
```

LE CLAVIER NE CACHE PAS LE CHAMP

```
<KeyboardAvoidingView
  style={styles.container}
  behavior={Platform.OS === "ios" ? "padding" : "height"}
>
```

Quand le clavier monte, `KeyboardAvoidingView` décale le contenu pour que le champ actif reste visible. iOS se comporte mieux avec `padding`, Android avec `height`.

Compte ou événement ? `sInscrire` crée un compte, `inscrire` inscrit à un événement. Ici, on parle de compte. Le sujet suppose que la session arrive aussitôt après `sInscrire` ; sur ce projet, la confirmation d'email est active (Lab 6) : Supabase crée le compte mais ne rend pas de session tant que le lien n'est pas cliqué. L'écran le dit au lieu de rester muet :

```
if (inscription) {
  const { session } = await sInscrire(email.trim(), motDePasse, nom.trim());
  // Le projet exige la confirmation de l'email : le compte est cree,
  // mais Supabase ne rend pas de session tant que le lien n'est pas clique.
  if (!session) {
    setInfo(
      `Compte cree. Confirmez l'email envoye a ${email.trim()}, puis connectez-vous.`
    );
    setMode("login");
    setMotDePasse("");
  }
} else {
  await seConnecter(email.trim(), motDePasse);
}
// La session se met a jour toute seule via onAuthStateChange
```

Comme sur le web, le mode inscription demande aussi un nom d'affichage : il part dans les métadonnées du compte, où le trigger du Lab 5 le lit pour créer la ligne `profiles` (sans lui, le profil s'appellerait « Invite »). Les règles que le navigateur vérifiait seul (`required`, `minLength="6"`) sont vérifiées à la main : le parcours mesure 0 appel à `/auth/v1/signup` pour un mot de passe trop court.

04 Le bouton « + » et l'écran de création

Le bouton est un vrai composant React : il lit la session avec `useAuth` pour ne s'afficher que connecté, et navigue avec `useRouter`.

```
export default function BoutonAjouter() {
  const { session } = useAuth();
  const router = useRouter();

  if (!session) return null;
```

Le sujet le place dans `headerRight`, l'emplacement de droite de l'en-tête du navigateur. Mais l'onglet Événements masque cet en-tête depuis le Lab 9, pour garder l'en-tête de marque (dégradé, compteur) : un `headerRight` n'y serait jamais dessiné. `EnteteAccueil` reçoit donc une prop `action` qui joue le même rôle :

```
<EnteteAccueil
  nombre={evenements.length}
  chargement={chargement}
  action={<BoutonAjouter />}
```

L'écran `nouveau.js` appelle `creerEvenement()`. `organisateur_id` vaut l'id de l'utilisateur connecté : c'est ce qu'exige la policy `evenements_insert_auth` du Lab 5. Une fois l'insertion faite, `router.back()` ramène à la liste.

```
await creerEvenement({
  titre: titre.trim(),
  categorie,
  lieu_nom: lieu.trim(),
  date_debut: new Date().toISOString(),
  prix: Number(prix) || 0,
  organisateur_id: session.user.id,
});
router.back(); // retour a la liste, qui se recharge au focus
```

05 Des puces à la place du <select>

React Native n'a pas de liste déroulante simple. On mappe les catégories de `theme.js` en `Pressable`, et le style de la puce choisie se combine au style de base : `[styles.chip, actif && {…}]`, l'équivalent natif d'une classe conditionnelle. Chaque puce reprend l'icône et la couleur de sa catégorie, les mêmes que sur les cartes.

```
{CATEGORIES.map((c) => {
  const actif = categorie === c.cle;
  return (
    <Pressable
      key={c.cle}
      onPress={() => setCategorie(c.cle)}
      style={[
        styles.chip,
        actif && { backgroundColor: c.couleur, borderColor: c.couleur },
      ]}
    >
      <Ionicons name={c.icone} size={14} color={actif ? "#fff" : c.couleur} />
      <Text style={actif ? styles.chipTexteActif : styles.chipTexte}>
        {c.cle}
      </Text>
    </Pressable>
  );
})}
```

06 useFocusEffect : recharger au retour

Après `router.back()`, la liste réapparaît… sans le nouvel événement. Un onglet n'est pas démonté quand on le quitte : il reste en mémoire, et un `useEffect(fn, [])` ne repasse pas. `useFocusEffect` relance le chargement chaque fois que l'écran redevient visible ; `useCallback` évite de recréer la fonction à chaque rendu.

```
useFocusEffect(
  useCallback(() => {
    charger().finally(() => setChargement(false));
  }, [charger])
);
```

Le parcours le mesure : revenir sur l'onglet ajoute 1 requête `GET /evenements`, et la liste passe de 10 à 11 événements après la création. Au Lab 9, le même retour ne rechargeait rien : c'est le comportement voulu qui a changé. Les squelettes de chargement ne s'affichent qu'au premier passage ; les suivants remplacent la liste sans la vider.

07 S'inscrire à un événement

`BoutonInscription` est le composant web du Lab 6 traduit en natif : `Pressable` au lieu de `button`, `StyleSheet` au lieu de CSS, et la session lue dans le contexte au lieu d'une prop. La logique tient dans les trois fonctions partagées.

```
const basculer = async () => {
  setErreur(null);
  setEnCours(true);
  try {
    if (inscrit) {
      await desinscrire(evenementId, session.user.id);
      setInscrit(false);
    } else {
      await inscrire(evenementId, session.user.id);
      setInscrit(true);
    }
  } catch (e) {
    setErreur(e.message);
  } finally {
    setEnCours(false);
  }
};
```

La policy `inscriptions_insert_self` impose `utilisateur_id = auth.uid()` : on ne peut inscrire que soi-même, quoi qu'envoie le client. L'état `enCours` désactive le bouton pendant l'appel ; un double toucher aurait sinon tenté une seconde insertion, refusée par la contrainte `unique(evenement_id, utilisateur_id)`. La vérification `estInscrit` est rejouée à chaque changement de session : se connecter depuis l'onglet Profil, détail ouvert, met le bouton à jour.

08 Bonus : « Mes inscriptions »

Une fonction de lecture ajoutée au package partagé. Supabase suit la clé étrangère `evenement_id` et ramène le titre et la date de chaque événement dans la même requête.

PACKAGES/SHARED/SRC/INSCRIPTIONS.TS

```
export async function mesInscriptions(utilisateurId: string) {
  const { data, error } = await getSupabase()
    .from("inscriptions")
    .select("evenement_id, evenements(titre, date_debut)")
    .eq("utilisateur_id", utilisateurId);
  if (error) throw error;
  return data ?? [];
}
```

Le sujet suggère de la charger dans un `useEffect`. Mais l'inscription se fait depuis l'écran de détail, pendant que l'onglet Profil reste monté en arrière-plan : c'est le piège de l'étape 3. La liste est donc relue avec `useFocusEffect` ; le parcours la voit passer de 1 à 2 en revenant sur l'onglet. Chaque ligne est un `Link` vers le détail de l'événement.

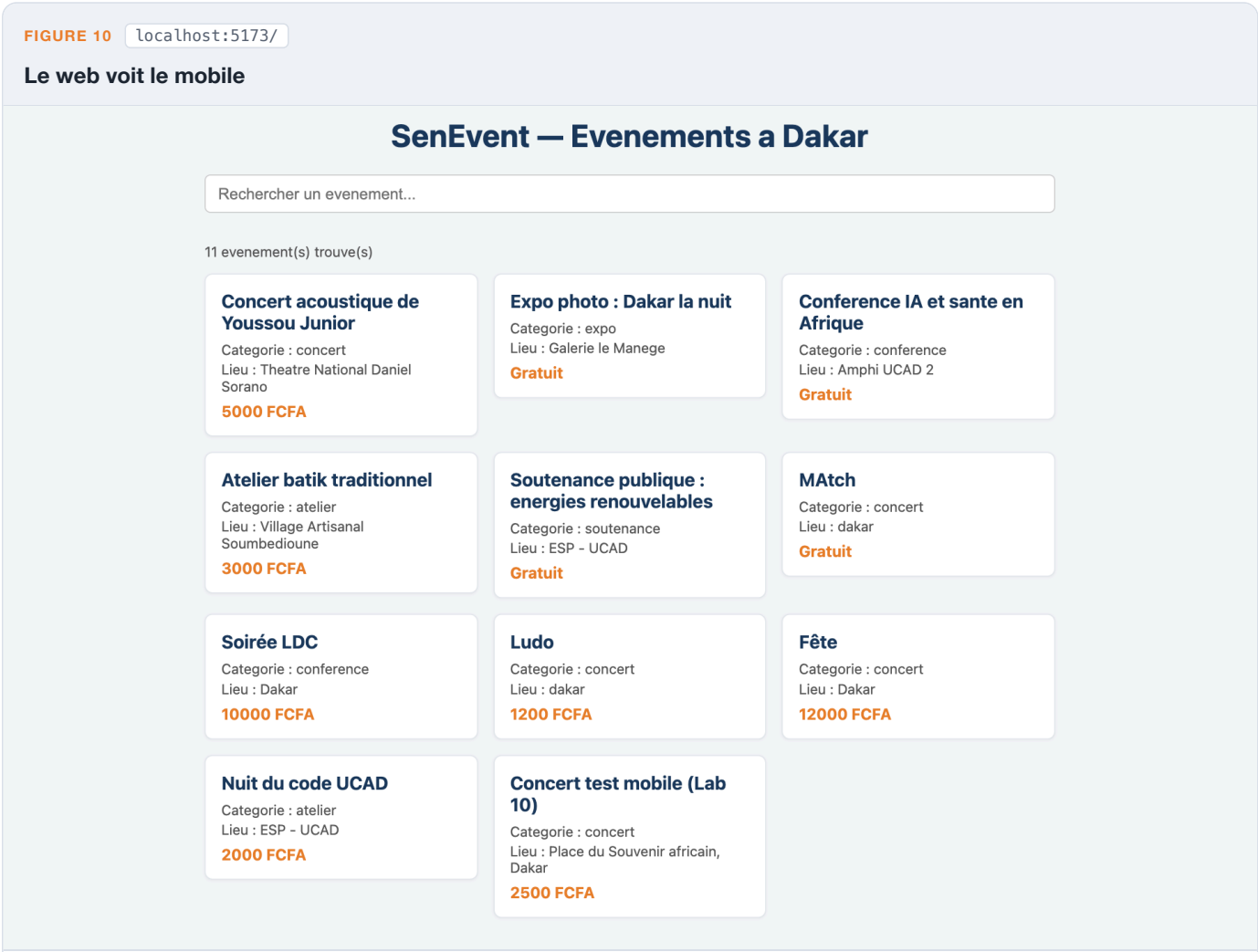
09 La démo croisée : deux clients, une base

L'expérience proposée par le sujet, faite avec les deux comptes de test du Lab 6 : Khady sur le site web, Atou sur le téléphone. L'app web du Lab 6 tourne sans modification de logique.

ETAPE DU SUJET	CE QUI S'EST PASSE
1. Sur le web, creer « Concert test »	`Khady (compte B) remplit le formulaire du Lab 6 : HTTP 201.`
2. Sur le mobile, revenir a la liste	`Atou passe sur l'onglet Profil puis revient : useFocusEffect relance getEvenements(), sans tirer pour rafraichir.`
3. « Concert test » apparait	`Figure 12 : organisateur Khady Ndiaye, lu par la jointure profiles(nom).`
4. S'y inscrire depuis le mobile	`inscrire() : HTTP 201 (figure 13).`
5. Sur le web, l'inscription est visible	`La table inscriptions contient 1 ligne pour cet evenement, au nom d'Atou ; connecte sur le site, Atou voit Se desinscrire (figure 14).`
Et dans l'autre sens	`L'evenement cree sur le telephone (HTTP 201) apparait dans le catalogue web et sur sa page de detail (figures 10 et 11).`

FIGURE 10 `localhost:5173/`

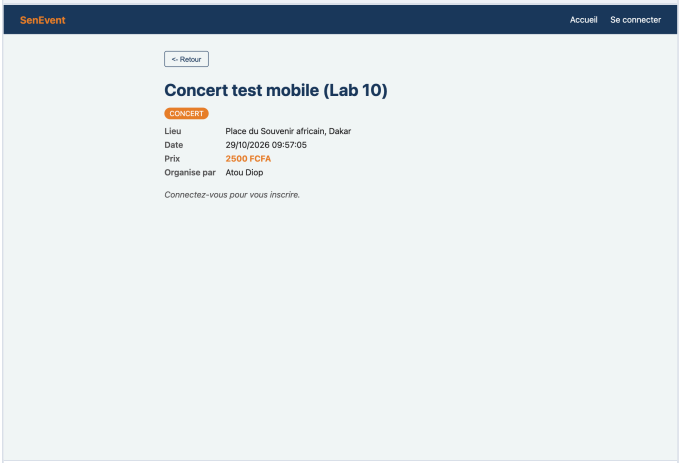
Le web voit le mobile



Le catalogue web du Lab 6, sans rien changer a son code : la carte « Concert test mobile (Lab 10) » creee depuis le telephone est la, en derniere position (tri par date).

FIGURE 11 `localhost:5173/evenement/15`

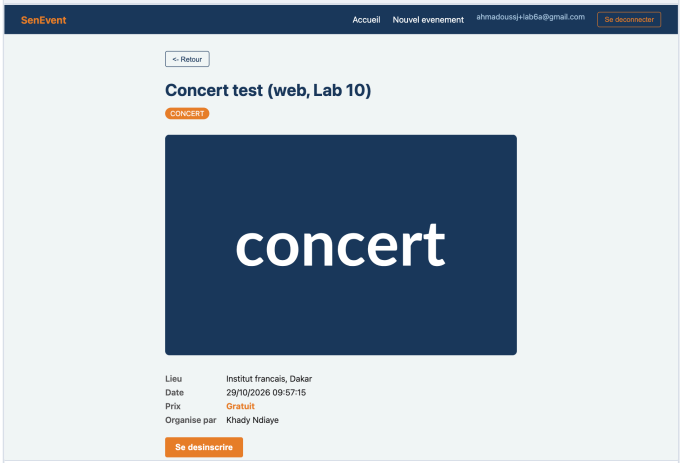
Le detail web de l'evenement mobile



Organise par Atou Diop : organisateur_id vient de session.user.id cote mobile. Sans affiche, la page n'affiche plus d'image cassee (correctif de ce lab).

FIGURE 14 `localhost:5173/evenement/16`

Le web voit l'inscription



Atou se connecte sur le site : pour l'evenement de Khady, le bouton est deja Se desinscrire. L'inscription a ete faite depuis le telephone.

Les deux événements de test ont ensuite été supprimés, chacun par son organisateur — seule personne autorisée par la policy `evenements_delete_owner`. Leurs inscriptions sont parties avec, par le `on delete cascade` du Lab 5 : la base est revenue à son état d'avant la démo.

10 Écarts assumés par rapport au sujet

Le code du sujet a été suivi pour la structure, les noms et la logique. Voici ce qui diffère, et pourquoi :

LE SUJET	ICI	POURQUOI
Styles clairs (#1a3a5c, #f0f4f8)	`theme sombre de theme.js`	`la charte des Labs 8 et 9 ; meme structure de composants, memes props`
`headerRight` dans (tabs)/_layout.js	`BoutonAjouter passe a EnteteAccueil par sa prop action`	`l'onglet masque l'en-tete du navigateur depuis le Lab 9 (headerShown: false) : un headerRight n'y serait jamais dessine`
`sInscrire(email, motDePasse)`	`un champ Nom, et le cas « pas de session »`	`le nom alimente le trigger du Lab 5 comme sur le web ; la confirmation d'email est active, signUp ne rend donc pas de session`
un message d'erreur unique	`erreurs par champ, effacees a la saisie`	`parite avec NouvelEvenement.jsx ; le prix doit etre un entier (colonne integer)`
rien si deconnecte sur /nouveau	`ecran « Connexion requise »`	`l'ecran reste joignable par un lien direct senevent://nouveau`
basculer sans protection	`try/catch et etat enCours`	`un double toucher violerait unique(evenement_id, utilisateur_id) ; parite avec le bouton web`
bonus avec `useEffect`	`useFocusEffect`	`l'onglet Profil reste monte : la lecon de l'etape 3 s'applique aussi a lui`
—	`web : pas de <img> si image_url est null`	`un evenement cree sur mobile n'a pas d'affiche ; le detail web affichait une image cassee`
—	`SDK 57 commite en tete de branche`	`la montee de version d'Expo faite apres le Lab 9 n'etait pas commitee ; le lab part d'un etat reproductible`

11 Structure livrée

```
apps/mobile/
  app/
    _layout.js              # Stack + AuthProvider + initSupabase (+ ecran nouveau)
    nouveau.js              # creation d'evenement : creerEvenement()
    (tabs)/
      _layout.js            # navigateur Tabs
      index.js              # liste : useFocusEffect + bouton "+"
      profil.js             # connexion / inscription / deconnexion + Mes inscriptions
    evenement/
      [id].js               # detail + BoutonInscription
  components/
    BoutonAjouter.js        # "+" visible seulement si connecte
    BoutonInscription.js    # estInscrit / inscrire / desinscrire
    EnteteAccueil.js        # en-tete de marque, emplacement "action"
    EvenementCarte.js · FiltreCategories.js · CarteSquelette.js
  context/
    AuthContext.js          # AuthProvider + useAuth
  theme.js · format.js · metro.config.js · app.json
packages/shared/src/
  inscriptions.ts           # + mesInscriptions (bonus)
```

12 Ce que la validation a réellement donné

Les vérifications de l'enseignant (section 9.2), le contrôle de types, le build web et les bundles natifs, relancés au moment de générer ce rapport :

CONTRÔLE	COMMANDE / MÉTHODE	RÉSULTAT	OK
Le contexte existe	`test -f apps/mobile/context/AuthContext.js`	OK	✓
Les ecrans lisent le contexte	`grep -rl "useAuth|AuthProvider" apps/mobile/app`	`app/(tabs)/profil.js · app/nouveau.js · app/_layout.js`	✓
Les actions passent par le package partage	`git grep -l "creerEvenement|inscrire|seConnecter"`	`app/(tabs)/profil.js · app/nouveau.js · components/BoutonInscription.js`	✓
Aucun appel Supabase direct (fichiers suivis)	`git grep "supabase.from|createClient" -- apps/mobile`	0 occurrence	✓
Aucun appel Supabase direct (commande du sujet)	`grep -rc "supabase.from|createClient" apps/mobile`	0 fichier non nul	✓
supabase-js jamais importe par les ecrans	`git grep "@supabase/supabase-js" -- app components context`	0 fichier	✓
Le .env mobile n'est pas versionne	`git ls-files | grep -c "apps/mobile/.env$"`	0 fichier suivi	✓
Types du package partage (bonus inclus)	`npx tsc --noEmit -p packages/shared`	aucune erreur	✓
Le web construit toujours	`npm run build (apps/web)`	build Vite reussi	✓
Bundle Android	`npx expo export --platform android`	1376 modules	✓
Bundle iOS	`npx expo export --platform ios`	1241 modules	✓
Une seule copie de React	`sources de la sourcemap Android`	`react : /node_modules · react-native : /node_modules`	✓

Les bundles natifs contiennent bien `AuthContext.js`, `nouveau.js` et `BoutonInscription.js` : le code du lab n'est pas propre à la cible web. Le lab compte 8 commits depuis `v0.9-router`, dans l'ordre de merge imposé : contexte et Profil (A), puis création (B) et inscription (C), puis le bonus.

Le parcours, piloté par `apps/mobile/capture-lab10.mjs` sur l'application réelle, la vraie base et l'app web à côté :

CONTRÔLE	MÉTHODE	RÉSULTAT	OK
Deconnecte : pas de bouton +	en-tete de la liste	absent	✓
Formulaire de connexion	champs de l'onglet Profil	Email · Mot de passe (type=password)	✓
Mode inscription : champ Nom en plus	basculer le formulaire	Nom d'affichage · Email · Mot de passe	✓
Mot de passe trop court arrete localement	appels /auth/v1/signup	0 appel · message affiche	✓
Mauvais mot de passe	seConnecter() refuse	HTTP 400 · Invalid login credentials	✓
Connexion	seConnecter() + onAuthStateChange	HTTP 200 · nom et email affiches	✓
Connecte : le bouton + apparait	en-tete de la liste	visible	✓
useFocusEffect recharge la liste	GET /evenements au retour sur l'onglet	+1	✓
Formulaire vide refuse localement	POST /evenements	0 requete · 2 messages	✓
Creation depuis le mobile	creerEvenement() puis router.back()	HTTP 201 · / · 10 → 11 evenements	✓
L'organisateur est le compte connecte	profiles(nom) de l'evenement cree	Atou Diop	✓
Le web voit l'evenement du mobile	liste et detail de apps/web	organisateur affiche · image cassee : non	✓
Khady cree un evenement sur le web	formulaire du Lab 6	HTTP 201	✓
Le mobile le voit au retour sur la liste	GET /evenements, aucune action manuelle	+1 · organisateur Khady Ndiaye	✓
Inscription depuis le mobile	inscrire() puis lecture directe de la table	HTTP 201 · 1 ligne au nom d'Atou	✓
Le web voit l'inscription	detail web, connecte en Atou	Se desinscrire	✓
Bonus : Mes inscriptions	mesInscriptions() au focus du Profil	1 → 2	✓
Deconnexion	seDeconnecter()	HTTP 204 · formulaire revenu · + absent	✓
Detail deconnecte	BoutonInscription sans session	invite a se connecter	✓
Lien direct vers /nouveau, deconnecte	ecran protege puis bouton	Connexion requise → /profil	✓
Nettoyage des donnees de test	DELETE par chaque organisateur (RLS)	1 + 1 evenement supprime	✓
Aucune erreur JavaScript	pageerror, web et mobile	0 erreur	✓

13 Limites connues

La session ne survit pas à la fermeture de l'app. `initSupabase` crée le client sans option de stockage. Dans un navigateur, supabase-js range alors la session dans `localStorage` ; sur un vrai téléphone, il n'y a pas de `localStorage` et il la garde en mémoire. Dans Expo Go, il faut donc se reconnecter après avoir fermé l'application. La correction classique est de passer `AsyncStorage` en option `auth.storage` — utile pour l'APK du Lab 12.

Les messages d'erreur de Supabase restent en anglais (« Invalid login credentials »), comme sur le web : les deux affichent `e.message` tel quel.

La date d'un événement créé est l'instant de la création, comme sur le web. Un vrai sélecteur de date demanderait un composant natif supplémentaire.

14 L'application, en situation

Captures au format téléphone, sur la cible web d'Expo — le même dossier `app/` que celui exécuté dans Expo Go, rendu dans un navigateur pour pouvoir piloter le parcours par script. Le chemin de chaque écran est indiqué en tête de figure.

FIGURE 01 `/`

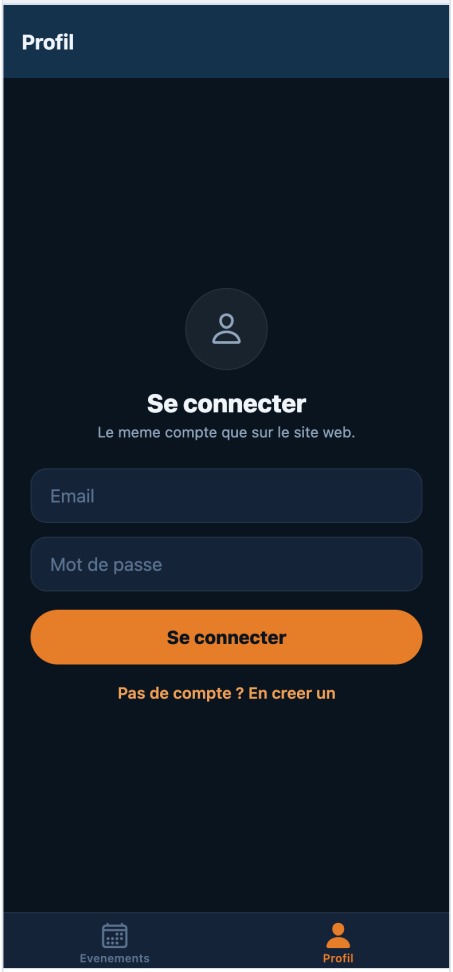
Deconnecte



La liste du Lab 9, 10 evenements. Sans session, BoutonAjouter rend null : l'en-tete n'a pas de bouton +.

FIGURE 02 `/profil`

Formulaire de connexion



L'ancien placeholder est remplace par FormulaireAuth : deux TextInput, le mot de passe masque par secureTextEntry.

FIGURE 03 `/profil`

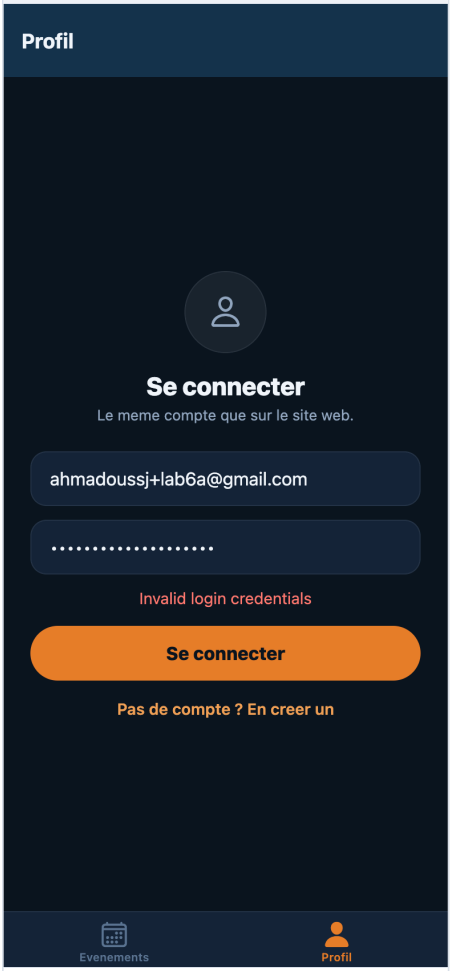
Creer un compte



Le mode inscription ajoute le nom d'affichage, comme sur le web. Un mot de passe de 3 caracteres est refuse avant tout appel reseau.

FIGURE 04 `/profil`

Mauvais mot de passe



Supabase repond 400 ; seConnecter() leve l'erreur, le formulaire l'affiche tel quel.

FIGURE 05 `/profil`

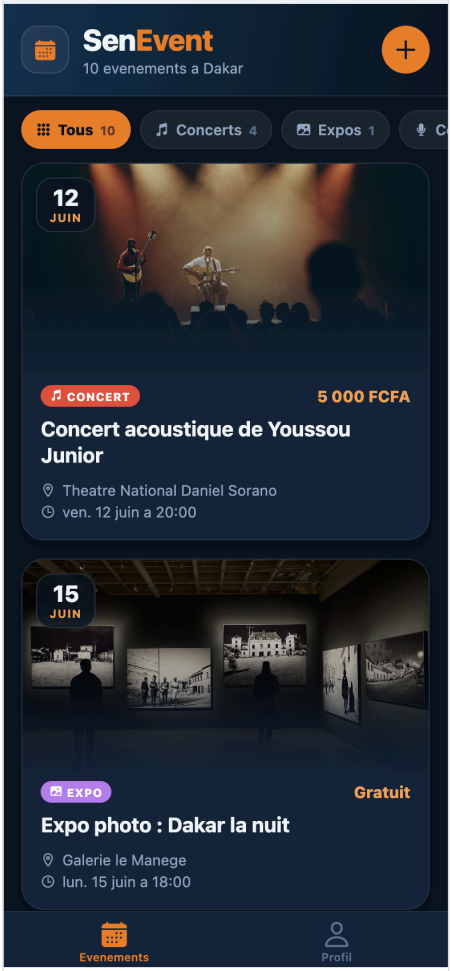
Connecte



onAuthStateChange a mis la session a jour dans le contexte : le formulaire laisse place au compte, sans navigation ni rechargement.

FIGURE 06 `/`

Le bouton + apparait



Le meme en-tete, une fois connecte. BoutonAjouter lit la session avec useAuth et pousse /nouveau avec useRouter.

FIGURE 07 /nouveau

Formulaire vide



Les regles du formulaire web, verifiees avant l'envoi : aucune requete n'est partie.

FIGURE 08 /nouveau

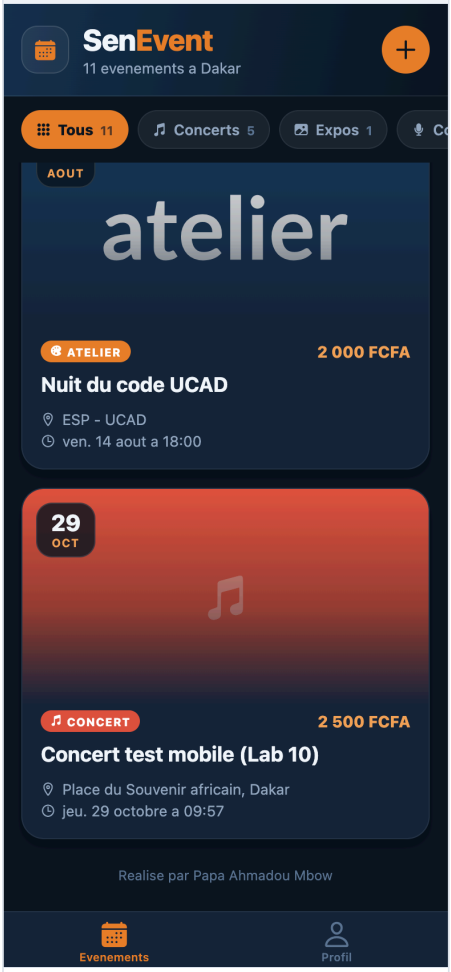
Formulaire rempli



Les puces de categorie remplacent le <select> du web ; le prix se saisit au pave numerique.

FIGURE 09 /

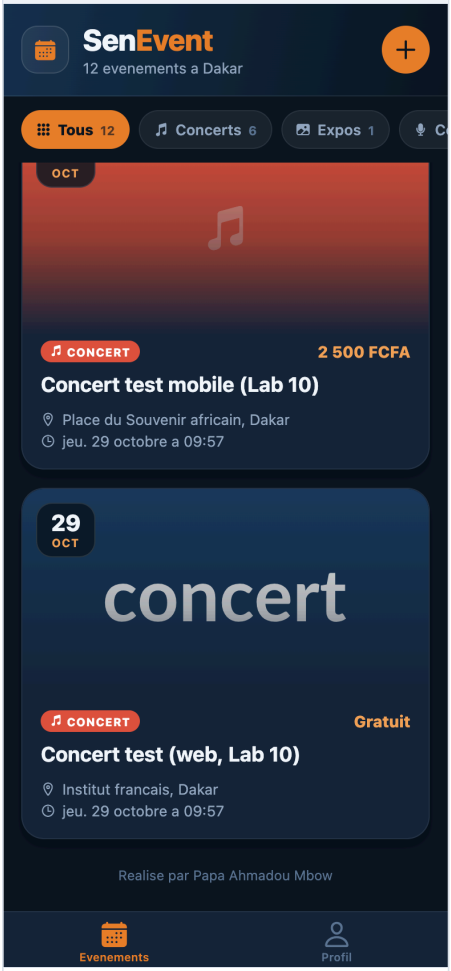
Retour a la liste



router.back(), puis useFocusEffect recharge : 10 → 11 evenements. Sans affiche, la carte prend la couleur de sa categorie.

FIGURE 12 `/`

Le mobile voit le web



L'evenement de Khady, cree sur le site, apparait des le retour sur l'onglet (+1 requete).

FIGURE 13 `/evenement/16`

Inscription depuis le telephone



inscrire() : le bouton passe a Se desinscrire. La ligne est relue directement dans la table inscriptions.

FIGURE 15 `/profil`

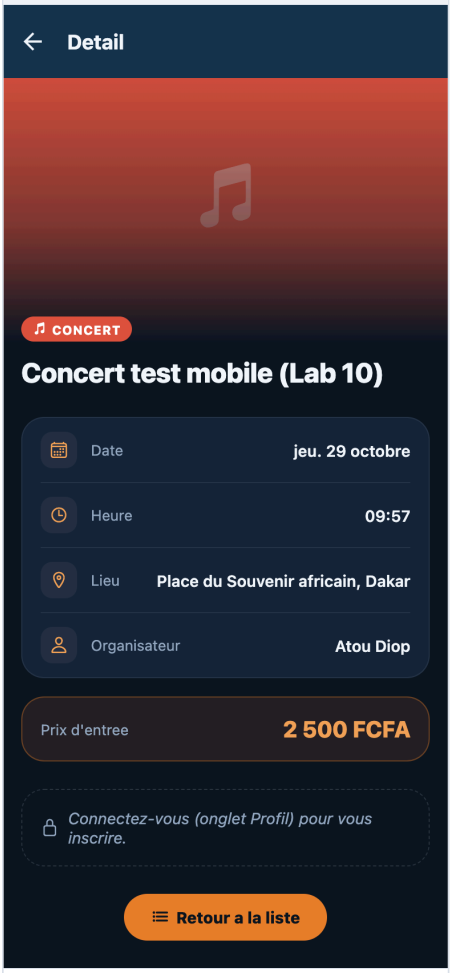
Bonus : Mes inscriptions



mesInscriptions() relue au focus du Profil : 1 → 2. Toucher une ligne ouvre son detail.

FIGURE 16 `/evenement/15`

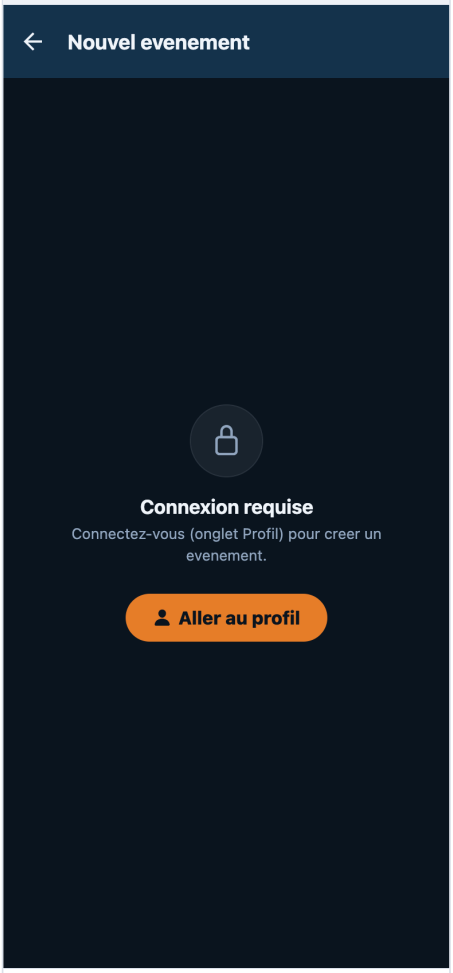
Detail, deconnecte



Apres seDeconnecter(), BoutonInscription invite a se connecter ; le toucher mene a l'onglet Profil.

FIGURE 17 `/nouveau`

Lien direct, deconnecte



L'ecran de creation reste joignable par senevent://nouveau : il refuse d'afficher un formulaire que la RLS rejetterait.

Vers le Lab 11

Web et mobile font désormais la même chose, sur la même base : c'est la fin du développement. Mais tout tourne encore en local — le web sur `localhost:5173`, le mobile par Expo Go sur le réseau de l'équipe. Le Lab 11 met l'application web en ligne sur Vercel, avec ses variables d'environnement côté hébergeur ; le Lab 12 construira l'APK Android avec EAS Build.